scConform vignette

Daniela Corbetta

2025-12-18

Contents

Introduction **1**

Installation **2**

Preliminaries **2**
- Setup . . . 2
- Load data and cell ontology . . . 2
- Build the ontology . . . 3
- Preprocess data . . . 4
- Fit a classification model . . . 6

Method 1: standard conformal inference **7**
- Algorithm . . . 7
- Obtain prediction matrices . . . 7
- Obtain conformal prediction sets . . . 8

Method 2: exploit the cell ontology **10**
- Algorithm . . . 10
- Obtain hierarchical prediction sets . . . 11

Visualization **12**

R.session Info **15**

Introduction

Cell type annotation is a crucial step in bioinformatic research. A multitude of annotated datasets are now readily accessible, providing valuable references for annotating cells in unannotated datasets originating from similar tissues. Typically, a model is chosen and trained on the reference data to predict the label of a new, unannotated cell in the query dataset.

These methods commonly provide point predictions of the cell label, along with estimated probabilities or scores assigned to each cell type in the reference dataset. However, relying solely on point predictions can be problematic when the corresponding estimated probability is low, leading to unreliable classification. To address this issue, we propose to exploit conformal inference to return prediction sets that include multiple labels, with the set size reflecting the confidence in the point prediction.

The package `scConform` implements two methods to achieve this:

1. The first method uses split conformal inference (as a reference, see for example section 1 of Angelopoulus and Bates, 2022). The output is a prediction set, $C(X_{new})$, that contains some of the cell types present

in the reference data. Let Y_{new} be the true cell type of a new cell in the query dataset. Then, $C(X_{new})$ satisfies $P(Y_{new} \in C(X_{new})) \geq 1 - \alpha$, where α is a user-chosen error level.

2. The second method is based on conformal risk control (Angelopoulus et al., 2023). It considers the inherent relationships among cell types that are encoded as graph-structured constraints available through the cell ontology. The final output is a prediction set aligned with the cell ontology structure. Specifically, the prediction set will include all the children of an ancestor of the predicted class. The more unsure the point prediction, the broader the classification. Also this method satisfies $P(Y_{new} \in C(X_{new})) \geq 1 - \alpha$.

Installation

To install it

```
if (BiocManager::version() >= "3.20") {
    BiocManager::install("scConform")
} else {
    spdl::info(
        "'scConform' requires Bioconductor version 3.20 or later, ",
        "installing development version from Github"
    )
    devtools::install_github("ccb-hms/scConform")
}
```

Preliminaries

Setup

```
library(scConform)
library(SingleCellExperiment)
library(VGAM)
library(ontoProc)
library(MerfishData)
library(igraph)
library(scuttle)
library(scran)
library(BiocParallel)

`%notin%` <- Negate(`%in%`)
```

Load data and cell ontology

As an example, we'll use the mouse ileum Merfish data from the `MerfishData` Bioconductor package, segmented with Baysor. For the purpose of this vignette, we'll treat the data as if they were single-cell data, discarding the spatial information. This is a toy example in which reference and query data will be a subset of the same dataset. In real life examples, data integration is a necessary first step.

Next, load the cell ontology trough the `ontoProc` Bioconductor package.

```
# Load data
spe_baysor <- MouseIleumPetukhov2021(
    segmentation = "baysor",
    use.images = FALSE, use.polygons = FALSE
)
```

```r
# Load ontology
cl <- getOnto("cellOnto", "2023")
#> loading from cache
```

Build the ontology

The object `cl` contains information regarding all the relationships among all the known cell types. The first step is to filter the ontology and only keep the parts related to the cell types that are present in the data.

This dataset does not provide annotations with the cell ontology tags, so we need to browse the ontology to find the interesting tags.

For more information on how to restrict the cell ontology, refer to the ontoProc documentation.

```r
tags <- c(
    "CL:0009022", # Stromal
    "CL:0000236", # B cell
    "CL:0009080", # Tuft
    "CL:1000411", # Endothelial
    "CL:1000335", # Enterocyte
    "CL:1000326", # Goblet
    "CL:0002088", # ICC
    "CL:0009007", # Macrophage + DC
    "CL:1000343", # Paneth
    "CL:0000669", # Pericyte
    "CL:1000278", # Smooth Muscle
    "CL:0009017", # Stem + TA
    "CL:0000492", # T (CD4+)
    "CL:0000625", # T (CD8+)
    "CL:0017004" # Telocyte
)
opi <- graph_from_graphnel(onto_plot2(cl, tags))
```

In the `opi` object, there are also instances from other ontologies (CARO and BFO) that need to be removed.

Moreover, the names of the leaf nodes of the ontology must correspond to the names used for the annotation.

```r
## Delete CARO and BFO instances
sel_ver <- V(opi)$name[c(grep("CARO", V(opi)$name), grep("BFO", V(opi)$name))]
opi1 <- opi - sel_ver

## Rename vertex to match annotations
V(opi1)$name[grep("CL:0000236", V(opi1)$name)] <- "B cell"
V(opi1)$name[grep("CL:1000411", V(opi1)$name)] <- "Endothelial"
V(opi1)$name[grep("CL:1000335", V(opi1)$name)] <- "Enterocyte"
V(opi1)$name[grep("CL:1000326", V(opi1)$name)] <- "Goblet"
V(opi1)$name[grep("CL:0002088", V(opi1)$name)] <- "ICC"
V(opi1)$name[grep("CL:0009007", V(opi1)$name)] <- "Macrophage + DC"
V(opi1)$name[grep("CL:1000343", V(opi1)$name)] <- "Paneth"
V(opi1)$name[grep("CL:0000669", V(opi1)$name)] <- "Pericyte"
V(opi1)$name[grep("CL:1000278", V(opi1)$name)] <- "Smooth Muscle"
V(opi1)$name[grep("CL:0009017", V(opi1)$name)] <- "Stem + TA"
V(opi1)$name[grep("CL:0009022", V(opi1)$name)] <- "Stromal"
V(opi1)$name[grep("CL:0000492", V(opi1)$name)] <- "T (CD4+)"
V(opi1)$name[grep("CL:0000625", V(opi1)$name)] <- "T (CD8+)"
V(opi1)$name[grep("CL:0017004", V(opi1)$name)] <- "Telocyte"
```

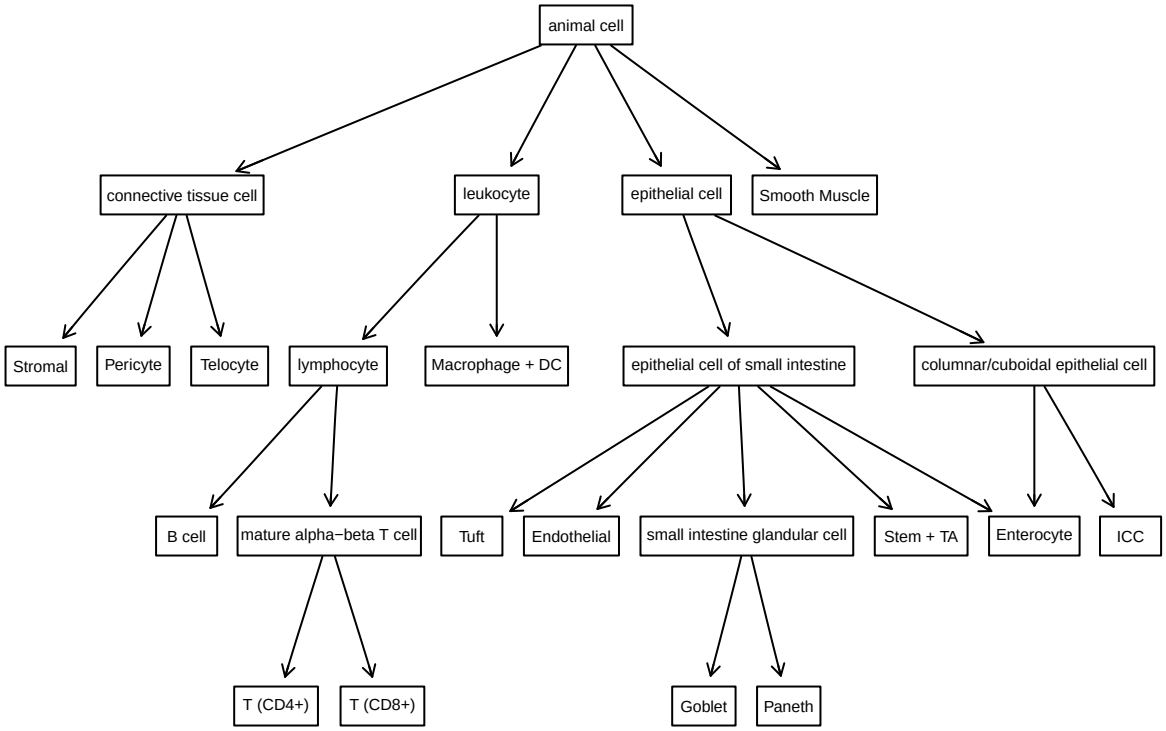
```r
V(opi1)$name[grep("CL:0009080", V(opi1)$name)] <- "Tuft"

## Add the edge from connective tissue cell and telocyte and delete redundant
## nodes
opi1 <- add_edges(opi1, c("connective\ntissue cell\nCL:0002320", "Telocyte"))
gr <- as_graphnel(opi1)
opi2 <- opi1 - c(
    "somatic\ncell\nCL:0002371", "contractile\ncell\nCL:0000183",
    "native\ncell\nCL:0000003"
)


V(opi2)$name <- trimws(gsub("CL:.*|\\n", " ", V(opi2)$name))

gr1 <- as_graphnel(opi2)

## Plot the final ontology
attrs <- list(node = list(shape = "box", fontsize = 15, fixedsize = FALSE))
plot(gr1, attrs = attrs)
```



Preprocess data

Modify the colData variable `leiden_final` to unify B cells and enterocytes

```r
spe_baysor$cell_type <- spe_baysor$leiden_final
spe_baysor$cell_type[spe_baysor$cell_type %in% c(
    "B (Follicular, Circulating)",
    "B (Plasma)"
)] <- "B cell"
spe_baysor$cell_type[grep("Enterocyte", spe_baysor$cell_type)] <- "Enterocyte"
```

```r
spe_baysor <- spe_baysor[, spe_baysor$cell_type %notin% c(
    "Removed",
    "Myenteric Plexus"
)]
spe_baysor
#> class: SpatialExperiment
#> dim: 241 5163
#> metadata(0):
#> assays(2): counts molecules
#> rownames(241): Acsl1 Acta2 ... Vcan Vim
#> rowData names(0):
#> colnames: NULL
#> colData names(8): n_transcripts density ... sample_id cell_type
#> reducedDimNames(0):
#> mainExpName: NULL
#> altExpNames(0):
#> spatialCoords names(2) : x y
#> imgData names(1): sample_id

# See frequencies of cell types
table(spe_baysor$cell_type)
#>
#>          B cell     Endothelial      Enterocyte          Goblet             ICC
#>             536             231            1257             299              31
#> Macrophage + DC          Paneth        Pericyte   Smooth Muscle       Stem + TA
#>             427             328             102             428             580
#>         Stromal        T (CD4+)        T (CD8+)        Telocyte            Tuft
#>             489             197             125             115              18
```

Now randomly split the data into reference and query data.

```r
set.seed(1636)
ref <- sample(seq_len(ncol(spe_baysor)), 600)

spe_ref <- spe_baysor[, ref]
spe_query <- spe_baysor[, -ref]
# Reference data
spe_ref
#> class: SpatialExperiment
#> dim: 241 600
#> metadata(0):
#> assays(2): counts molecules
#> rownames(241): Acsl1 Acta2 ... Vcan Vim
#> rowData names(0):
#> colnames: NULL
#> colData names(8): n_transcripts density ... sample_id cell_type
#> reducedDimNames(0):
#> mainExpName: NULL
#> altExpNames(0):
#> spatialCoords names(2) : x y
#> imgData names(1): sample_id
# Query data
spe_query
#> class: SpatialExperiment
```

```
#> dim: 241 4563
#> metadata(0):
#> assays(2): counts molecules
#> rownames(241): Acsl1 Acta2 ... Vcan Vim
#> rowData names(0):
#> colnames: NULL
#> colData names(8): n_transcripts density ... sample_id cell_type
#> reducedDimNames(0):
#> mainExpName: NULL
#> altExpNames(0):
#> spatialCoords names(2) : x y
#> imgData names(1): sample_id
```

Fit a classification model

We now need to build a classification model. To this aim, we'll draw a random sample of 300 cells from the reference data, and train a multinomial model on those. As explanatory variables we'll use the 50 most variable genes. For this step, every statistical model or machine learning method can be used, as long as it provides also estimated probabilities for each class.

```
# Randomly select 300 cells
set.seed(1704)
train <- sample(seq_len(ncol(spe_ref)), 300)
# Training data
spe_train <- spe_ref[, train]
spe_train
#> class: SpatialExperiment
#> dim: 241 300
#> metadata(0):
#> assays(2): counts molecules
#> rownames(241): Acsl1 Acta2 ... Vcan Vim
#> rowData names(0):
#> colnames: NULL
#> colData names(8): n_transcripts density ... sample_id cell_type
#> reducedDimNames(0):
#> mainExpName: NULL
#> altExpNames(0):
#> spatialCoords names(2) : x y
#> imgData names(1): sample_id

# get HVGs
spe_train <- logNormCounts(spe_train)
v <- modelGeneVar(spe_train)
hvg <- getTopHVGs(v, n = 50)

# Extract counts and convert data into a data.frame format
df_train <- as.data.frame(t(as.matrix(counts(spe_train[hvg, ]))))
df_train$Y <- spe_train$cell_type
table(df_train$Y)
#>
#>          B cell     Endothelial      Enterocyte          Goblet             ICC
#>              25              10              68              19               2
#> Macrophage + DC          Paneth        Pericyte   Smooth Muscle       Stem + TA
#>              28              17               4              22              48
```

```
#>         Stromal        T (CD4+)        T (CD8+)        Telocyte            Tuft
#>              34              13               5               4               1

# Fit multinomial model
fit <- vglm(Y ~ .,
    family = multinomial(refLevel = "B cell"),
    data = df_train
)
```

Method 1: standard conformal inference

Algorithm

As anticipated in the introduction, the first method uses split conformal inference to build the prediction sets. It requires to split the reference data into two subsets:

- training set: portion of the reference data to be used to build a classifier $\hat{f}$ (the multinomial model in our example)
- calibration set: portion of the reference data to be used to calibrate the procedure.

Given $\hat{f}$ and the calibration data, the objective of conformal inference is to build, for a new non-annotated cell X_{new}, a prediction set $C(X_{new})$ that satisfies

$$P(Y_{new} \in C(X_{new})) \geq 1 - \alpha, (\#eq : cov) \tag{1}$$

where Y_{new} is the true label of the new cell and α is a user-chosen error level. The algorithm for split conformal inference is the following:

1. For the data in the calibration set, $(X_1, Y_1), \ldots, (X_n, Y_n)$, obtain the conformal scores $s_1 = 1 - \hat{f}(X_i)_{Y_i}$ (i.e. 1 minus the estimated probability that the model is assigning to the true label of the i-th cell). These scores will be high when the model is assigning a small probability to the true class, and low otherwise.
2. Obtain $\hat{q}$ as the $\lceil (1-\alpha)(n+1) \rceil / n$ empirical quantile of the conformal scores.
3. Finally, for a new cell X_{new}, build a prediction set by including all the classes or which the estimated probability is higher than $1 - \hat{q}$:

$$C(X_{new}) = \left\{ y : \hat{f}(X_{n+1})_y \geq 1 - \hat{q} \right\}$$

Obtain prediction matrices

The first step to build conformal prediction sets is to obtain matrices with the estimated probabilities for each cell type for cells in the calibration data and in the query data. Each row of the matrix corresponds to a particular cell, while each row to a different cell type. The entry $p_{i,j}$ of the matrix indicates the estimated probability that the cell i is of type j.

```
spe_cal <- spe_ref[, -train]
# Prediction matrix for calibration data
df_cal <- as.data.frame(t(as.matrix(counts(spe_cal[hvg, ]))))
p_cal <- predict(fit, newdata = df_cal, type = "response")
head(round(p_cal, 3))
#>   B cell Endothelial Enterocyte Goblet   ICC Macrophage + DC Paneth Pericyte
#> 1  0.001       0.000      0.023  0.000 0.000           0.559  0.000    0.000
#> 2  0.000       0.000      0.000  0.000 0.000           0.000  0.000    0.000
```

```
#> 3  0.018       0.000      0.041  0.000 0.001           0.923  0.000    0.001
#> 4  0.001       0.001      0.281  0.035 0.001           0.003  0.017    0.000
#> 5  0.000       0.000      1.000  0.000 0.000           0.000  0.000    0.000
#> 6  0.000       0.000      0.002  0.000 0.000           0.000  0.141    0.000
#>   Smooth Muscle Stem + TA Stromal T (CD4+) T (CD8+) Telocyte  Tuft
#> 1         0.002     0.000   0.409    0.000    0.000    0.006 0.000
#> 2         0.000     0.000   0.000    0.001    0.999    0.000 0.000
#> 3         0.005     0.007   0.001    0.001    0.000    0.000 0.001
#> 4         0.004     0.161   0.489    0.001    0.002    0.003 0.001
#> 5         0.000     0.000   0.000    0.000    0.000    0.000 0.000
#> 6         0.000     0.856   0.000    0.000    0.000    0.000 0.000
# Prediction matrix for query data
df_test <- as.data.frame(t(as.matrix(counts(spe_query[hvg, ]))))
p_test <- predict(fit, newdata = df_test, type = "response")
head(round(p_test, 3))
#>   B cell Endothelial Enterocyte Goblet ICC Macrophage + DC Paneth Pericyte
#> 1      0       0.629          0      0   0               0      0        0
#> 2      0       0.000          0      0   0               0      0        0
#> 3      0       0.001          0      0   0               0      0        0
#> 4      0       0.000          0      0   0               0      0        0
#> 5      0       0.000          0      0   0               0      0        0
#> 6      0       0.000          0      0   0               0      0        0
#>   Smooth Muscle Stem + TA Stromal T (CD4+) T (CD8+) Telocyte Tuft
#> 1         0.000         0   0.001    0.000        0    0.370    0
#> 2         0.997         0   0.000    0.000        0    0.003    0
#> 3         0.021         0   0.002    0.001        0    0.975    0
#> 4         1.000         0   0.000    0.000        0    0.000    0
#> 5         0.929         0   0.069    0.000        0    0.001    0
#> 6         1.000         0   0.000    0.000        0    0.000    0
```

Obtain conformal prediction sets

We can now directly call the `getPredictionSet` function by using as input the prediction matrices for the calibration and the query dataset. In this case, the output of the function will be a list whose elements are the prediction sets for each query cell.

By setting `follow_ontology=FALSE`, we are asking the function to return prediction sets obtained via split conformal inference. The parameter `onto` is not necessary with this method, but if the ontology is provided then it will be used by the function to retrieve the considered cell types. Alternatively, we can explicitly provide the labels with the parameter `labels`. The parameter `alpha` indicates the allowed miscoverage percentage (see @ref(eq:cov)). For example, if we set `alpha=0.1`, it means that at most 10% of the prediction sets we obtain will not include the true label.

```
labels <- colnames(p_test)
sets <- getPredictionSets(
    x_query = p_test,
    x_cal = p_cal,
    y_cal = spe_cal$cell_type,
    alpha = 0.1,
    follow_ontology = FALSE,
    labels = labels
)

# See the first six prediction sets
```

```r
sets[1:6]
#> [[1]]
#> [1] "Endothelial" "Telocyte"
#>
#> [[2]]
#> [1] "Smooth Muscle"
#>
#> [[3]]
#> [1] "Smooth Muscle" "Telocyte"
#>
#> [[4]]
#> [1] "Smooth Muscle"
#>
#> [[5]]
#> [1] "Smooth Muscle" "Stromal"
#>
#> [[6]]
#> [1] "Smooth Muscle"
```

Since in this example the cell labels are available, we can explicitly check that @ref(eq:cov) is satisfied by computing the proportion of sets that include the true label. We set `alpha=0.1`, so we expect this proportion to be higher than 0.9.

```r
# Check coverage
cvg <- rep(NA, length(sets))
for (i in seq_len(length(sets))) {
    cvg[i] <- spe_query$cell_type[i] %in% sets[[i]]
}
mean(cvg)
#> [1] 0.9035722
```

As an alternative, we can provide as input a `SingleCellExperiment` object. In this case, it needs to have the estimated probabilities for each cell type in the `colData`. The names of these `colData` have to correspond to the names of the leaf nodes in the ontology.

The output will be a `SingleCellExperiment` object with a new variable in the `colData` containing the prediction sets. The name of this variable can be assigned trough the `pr_name` parameter.

```r
# Retrieve labels as leaf nodes of the ontology
labels <- V(opi2)$name[degree(opi2, mode = "out") == 0]

# Create corresponding colData
for (i in labels) {
    colData(spe_cal)[[i]] <- p_cal[, i]
    colData(spe_query)[[i]] <- p_test[, i]
}

# Create prediction sets
spe_query <- getPredictionSets(
    x_query = spe_query,
    x_cal = spe_cal,
    y_cal = spe_cal$cell_type,
    alpha = 0.1,
    follow_ontology = FALSE,
    pr_name = "pred_set",
    labels = labels
```

```
)

# See the new variable pred_set into the colData
head(colData(spe_query))
#> DataFrame with 6 rows and 24 columns
#>   n_transcripts   density elongation      area avg_confidence  leiden_final
#>       <numeric> <numeric>  <numeric> <numeric>      <numeric>   <character>
#> 1            39   0.02159      5.082      1806         0.8647   Endothelial
#> 2           165   0.02016      1.565      8186         0.9528 Smooth Muscle
#> 3           139   0.02279      1.820      6100         0.9762 Smooth Muscle
#> 4            80   0.01828      1.546      4376         0.9076 Smooth Muscle
#> 5            75   0.02479      3.475      3025         0.8952 Smooth Muscle
#> 6           167   0.02409      1.265      6932         0.9618 Smooth Muscle
#>     sample_id     cell_type     Stromal      B cell        Tuft Endothelial
#>   <character>   <character>   <numeric>   <numeric>   <numeric>   <numeric>
#> 1       ileum   Endothelial 1.07404e-03 4.74250e-06 1.11150e-06 6.28604e-01
#> 2       ileum Smooth Muscle 2.56252e-07 2.19266e-12 1.75805e-09 4.45708e-13
#> 3       ileum Smooth Muscle 1.84018e-03 1.07040e-06 3.44486e-05 1.46419e-03
#> 4       ileum Smooth Muscle 4.53652e-07 8.92427e-12 1.87252e-10 1.50607e-11
#> 5       ileum Smooth Muscle 6.93831e-02 5.47021e-05 3.86524e-05 1.58762e-04
#> 6       ileum Smooth Muscle 6.79811e-15 3.01443e-21 2.61053e-16 4.05051e-21
#>    Enterocyte      Goblet         ICC Macrophage + DC      Paneth    Pericyte
#>     <numeric>   <numeric>   <numeric>       <numeric>   <numeric>   <numeric>
#> 1 6.70650e-09 2.26557e-06 1.04210e-04     9.37672e-11 2.52820e-08 1.27673e-06
#> 2 4.06857e-15 5.86732e-19 3.36871e-10     3.89578e-15 1.07738e-11 8.11597e-11
#> 3 8.36949e-11 1.08169e-10 1.31288e-04     4.34569e-09 1.49813e-07 2.73419e-06
#> 4 3.21747e-10 4.47158e-16 8.92693e-10     1.04923e-11 3.43488e-09 1.79252e-11
#> 5 9.98919e-06 1.17618e-06 1.71533e-05     3.29898e-06 3.08033e-04 4.90776e-06
#> 6 1.42137e-21 1.07458e-37 8.04990e-16     3.40065e-21 1.16809e-26 5.95236e-22
#>   Smooth Muscle   Stem + TA    T (CD4+)    T (CD8+)    Telocyte
#>       <numeric>   <numeric>   <numeric>   <numeric>   <numeric>
#> 1   3.35824e-12 1.95335e-04 2.69517e-07 1.12077e-05 3.70001e-01
#> 2   9.97240e-01 4.68684e-18 3.15966e-10 4.13375e-10 2.75927e-03
#> 3   2.13831e-02 5.31276e-11 5.94206e-04 2.08193e-05 9.74528e-01
#> 4   9.99986e-01 5.83029e-10 4.60457e-09 9.24800e-10 1.36688e-05
#> 5   9.29279e-01 6.06252e-07 5.55908e-05 1.94503e-05 6.66065e-04
#> 6   9.99984e-01 1.84902e-24 1.76933e-14 5.08358e-15 1.64282e-05
#>                 pred_set
#>                   <list>
#> 1   Endothelial,Telocyte
#> 2          Smooth Muscle
#> 3 Smooth Muscle,Telocyte
#> 4          Smooth Muscle
#> 5  Stromal,Smooth Muscle
#> 6          Smooth Muscle
```

Method 2: exploit the cell ontology

Algorithm

Let $\hat{y}(x)$ be the class with the maximum estimated probability. Moreover, given a directed graph let $P(v)$ and $A(v)$ be the set of children nodes and ancestor nodes of v, respectively. Finally, for each node v define a score $g(v, x)$ as the sum of the predicted probabilities of the leaf nodes that are children of v. We build the

sets as follows:

$$P(v) \cup \{P(a) : a \in A(\hat{y}(x)) : g(a, x) \leq \lambda\},$$

where $v : v \in A(\hat{y}(x)),\ g(v, x) \geq \lambda$ and $v = \arg\min_{u:g(u,x)\geq\lambda} g(u, x)$.

In words, we start from the predicted class and we go up in the graph until we find an ancestor of $\hat{y}(x)$ that has a score that is at least λ and include in the prediction sets all its children. Then we add to this subgraph the other ones that contain $\hat{y}(x)$ for which the score is less than λ. To choose λ, we follow eq. (4) in Angelopoulus et al., 2023 (2022), considering the miscoverage as loss function.

Obtain hierarchical prediction sets

To switch to the hierarchical algorithm, we just need to set `follow_ontology=TRUE` and provide an ontology to the `onto` parameter. As in the previous case, if the input is a `SingleCellExperiment` object then the output will be again a `SingleCellExperiment` object with a new variable containing the prediction sets in the colData.

```
spe_query <- getPredictionSets(
    x_query = spe_query,
    x_cal = spe_cal,
    y_cal = spe_cal$cell_type,
    onto = opi2,
    alpha = 0.1,
    follow_ontology = TRUE,
    pr_name = "pred_set_hier",
    BPPARAM = MulticoreParam(workers = 2),
    simplify = FALSE
)

# See the first six prediction sets
head(spe_query$pred_set_hier)
#> [[1]]
#>  [1] "Tuft"            "Endothelial"     "Enterocyte"      "Goblet"
#>  [5] "ICC"             "Paneth"          "Stem + TA"       "Stromal"
#>  [9] "B cell"          "Macrophage + DC" "Pericyte"        "Smooth Muscle"
#> [13] "T (CD4+)"        "T (CD8+)"        "Telocyte"
#>
#> [[2]]
#> [1] "Smooth Muscle"
#>
#> [[3]]
#> [1] "Telocyte"
#>
#> [[4]]
#> [1] "Smooth Muscle"
#>
#> [[5]]
#> [1] "Smooth Muscle"
#>
#> [[6]]
#> [1] "Smooth Muscle"
```

Also in this case, we can check the coverage:

```r
# Check coverage
cvg1 <- rep(NA, length(spe_query$pred_set_hier))
for (i in seq_len(length(spe_query$pred_set_hier))) {
    cvg1[i] <- spe_query$cell_type[i] %in% spe_query$pred_set_hier[[i]]
}

mean(cvg1)
#> [1] 0.8915187
```

As an alternative, we can choose to return only the common ancestor, instead of the entire set of predicted labels. This can be done directly in the function by setting `simplify=TRUE`. Alternatively, we can convert the obtained prediction sets invoking the function `getCommonAncestor`.

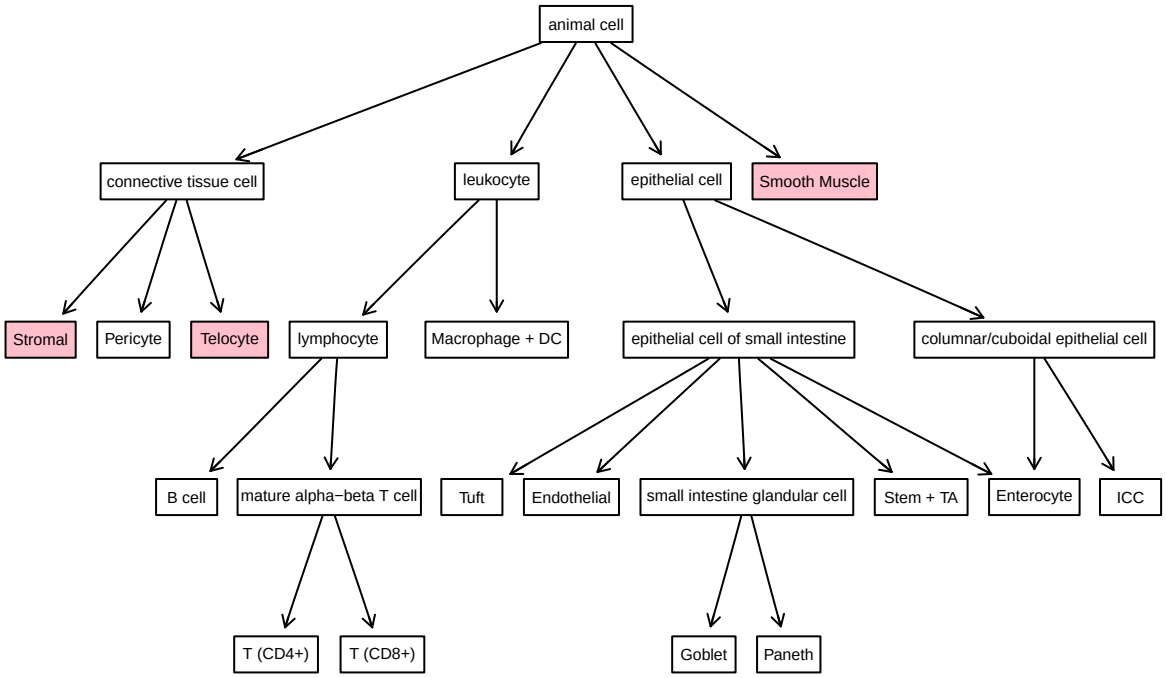
```r
spe_query$pred_set_hier_simp <- vapply(
    spe_query$pred_set_hier,
    function(x) getCommonAncestor(x, opi2),
    character(1)
)
head(spe_query$pred_set_hier_simp)
#> [1] "animal cell"   "Smooth Muscle" "Telocyte"      "Smooth Muscle"
#> [5] "Smooth Muscle" "Smooth Muscle"
```

Visualization

Let's now visualize the obtained prediction sets. The simplest way is to just ask to color the labels included in the prediction set. Let's compare the prediction sets for one cell obtained with the two methods

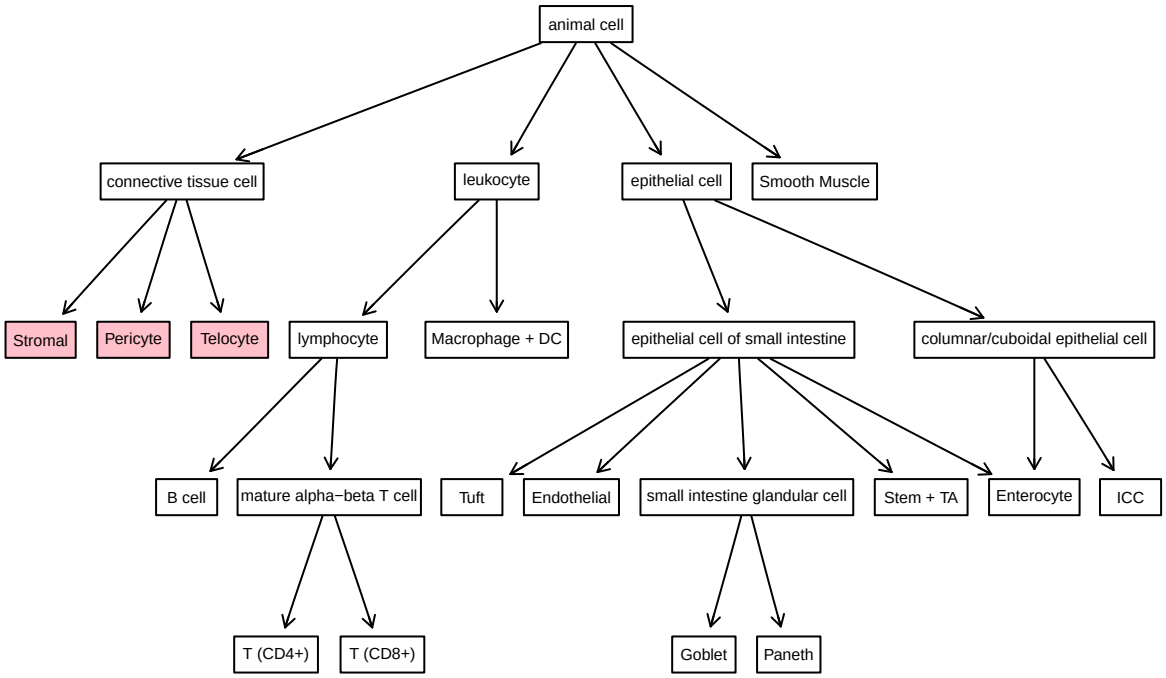
```r
plotResult(spe_query$pred_set[[75]], opi2,
    col_grad = "pink", attrs = attrs, add_scores = FALSE,
    title = "Conformal Prediction set"
)
```

Conformal Prediction set



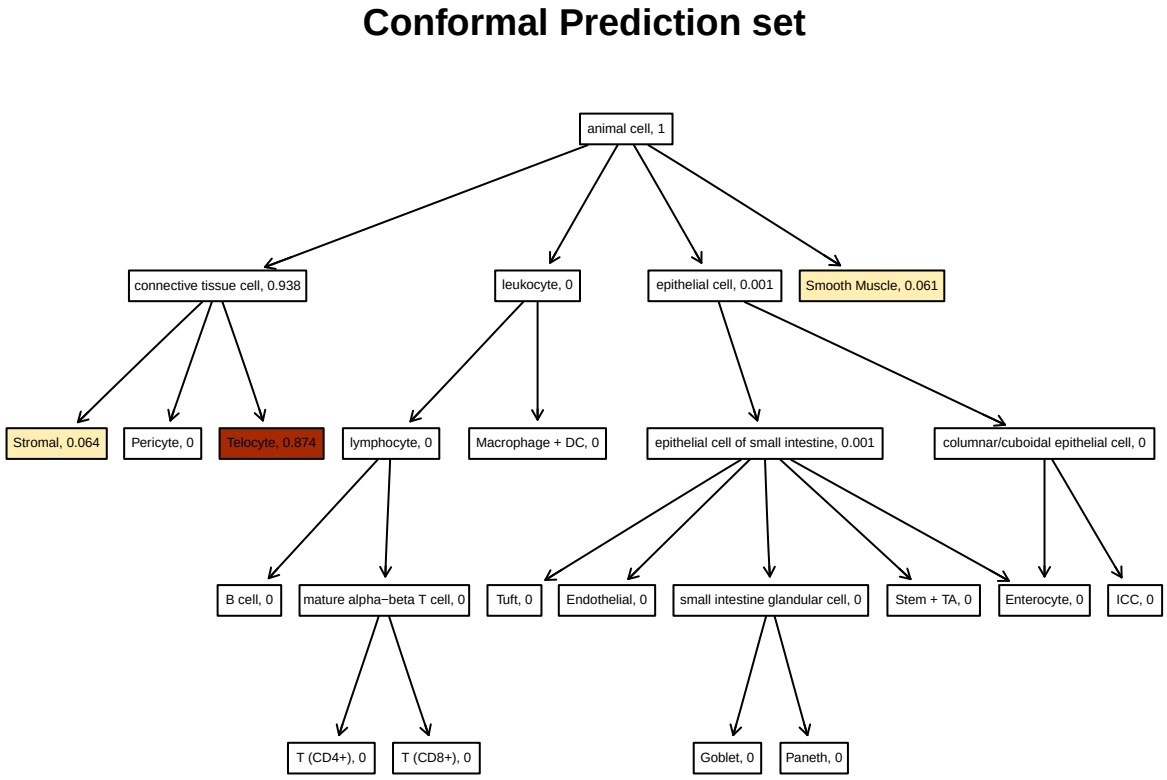

```r
plotResult(spe_query$pred_set_hier[[75]], opi2,
    col_grad = "pink", attrs = attrs, add_scores = FALSE,
    title = "Hierarchical prediction set"
)
```

Hierarchical prediction set



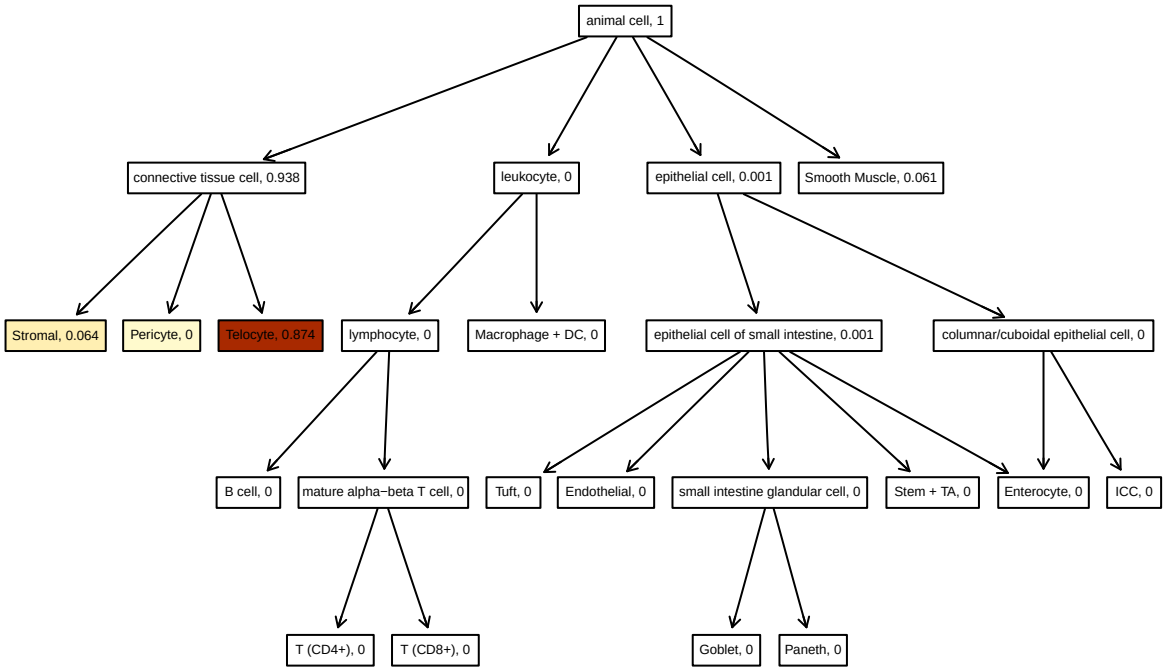
We can now instead give as input also the estimated probabilities and a gradient. The function will color the cells according to the gradient, with stronger colors corresponding to higher probabilities. If `add_scores=TRUE`, then the estimated probabilities will be added to the labels.

```
plotResult(spe_query$pred_set[[75]], opi2,
    probs = p_test[75, ],
    col_grad = c("lemonchiffon", "orange", "darkred"),
    attrs = attrs, add_scores = TRUE,
    title = "Conformal Prediction set"
)
```




```
plotResult(spe_query$pred_set_hier[[75]], opi2,
    probs = p_test[75, ],
    col_grad = c("lemonchiffon", "orange", "darkred"),
    attrs = attrs, add_scores = TRUE,
    title = "Hierarchical Prediction set"
)
```

Hierarchical Prediction set



R.session Info

```
R version 4.5.2 (2025-10-31)
Platform: aarch64-apple-darwin20
Running under: macOS Sonoma 14.7.6

Matrix products: default
BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK versi

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: Europe/Rome
tzcode source: internal

attached base packages:
[1] splines   stats4    stats     graphics  grDevices utils     datasets
[8] methods   base

other attached packages:
 [1] BiocParallel_1.44.0         scran_1.38.0
 [3] scuttle_1.20.0              igraph_2.2.1
 [5] MerfishData_1.12.0          SpatialExperiment_1.20.0
 [7] EBImage_4.52.0              ontoProc_2.4.0
 [9] ontologyIndex_2.12          VGAM_1.1-14
[11] SingleCellExperiment_1.32.0 SummarizedExperiment_1.40.0
[13] Biobase_2.70.0              GenomicRanges_1.62.1
```

```
[15] Seqinfo_1.0.0               IRanges_2.44.0
[17] S4Vectors_0.48.0            BiocGenerics_0.56.0
[19] generics_0.1.4              MatrixGenerics_1.22.0
[21] matrixStats_1.5.0           scConform_0.99.1

loaded via a namespace (and not attached):
  [1] rstudioapi_0.17.1    jsonlite_2.0.0       magrittr_2.0.4
  [4] magick_2.9.0         rmarkdown_2.30       vctrs_0.6.5
  [7] memoise_2.0.1        ellmer_0.4.0         RCurl_1.98-1.17
 [10] tinytex_0.58         htmltools_0.5.9      S4Arrays_1.10.1
 [13] AnnotationHub_4.0.0  curl_7.0.0           BiocNeighbors_2.4.0
 [16] Rhdf5lib_1.32.0      SparseArray_1.10.6   rhdf5_2.54.1
 [19] htmlwidgets_1.6.4    httr2_1.2.2          cachem_1.1.0
 [22] mime_0.13            lifecycle_1.0.4      pkgconfig_2.0.3
 [25] rsvd_1.0.5           Matrix_1.7-4         R6_2.6.1
 [28] fastmap_1.2.0        shiny_1.12.1         digest_0.6.39
 [31] AnnotationDbi_1.72.0 dqrng_0.4.1          irlba_2.3.5.1
 [34] ExperimentHub_3.0.0  RSQLite_2.4.5        beachmat_2.26.0
 [37] filelock_1.0.3       httr_1.4.7           abind_1.4-8
 [40] compiler_4.5.2       bit64_4.6.0-1        withr_3.0.2
 [43] S7_0.2.1             tiff_0.1-12          DBI_1.2.3
 [46] HDF5Array_1.38.0     rappdirs_0.3.3       DelayedArray_0.36.0
 [49] rjson_0.2.23         bluster_1.20.0       tools_4.5.2
 [52] otel_0.2.0           httpuv_1.6.16        glue_1.8.0
 [55] h5mread_1.2.1        rhdf5filters_1.22.0  promises_1.5.0
 [58] grid_4.5.2           cluster_2.1.8.1      BiocSingular_1.26.0
 [61] ScaledMatrix_1.18.0  metapod_1.18.0       XVector_0.50.0
 [64] BiocVersion_3.22.0   pillar_1.11.1        limma_3.66.0
 [67] BumpyMatrix_1.18.0   later_1.4.4          dplyr_1.1.4
 [70] BiocFileCache_3.0.0  lattice_0.22-7       bit_4.6.0
 [73] tidyselect_1.2.1     RBGL_1.86.0          coro_1.1.0
 [76] locfit_1.5-9.12      Biostrings_2.78.0    knitr_1.50
 [79] edgeR_4.8.1          ontologyPlot_1.7     xfun_0.55
 [82] statmod_1.5.1        DT_0.34.0            fftwtools_0.9-11
 [85] paintmap_1.0         yaml_2.3.12          evaluate_1.0.5
 [88] codetools_0.2-20     tibble_3.3.0         Rgraphviz_2.54.0
 [91] BiocManager_1.30.27  graph_1.88.1         cli_3.6.5
 [94] xtable_1.8-4         reticulate_1.44.1    Rcpp_1.1.0
 [97] dbplyr_2.5.1         png_0.1-8            parallel_4.5.2
[100] blob_1.2.4           jpeg_0.1-11          bitops_1.0-9
[103] purrr_1.2.0          crayon_1.5.3         rlang_1.1.6
[106] KEGGREST_1.50.0
```